

SISTEMA DE FACTURACIÓN

Gestiona, vende y crece sin límites...

Por Andrés Socorro

AS



¿En que consiste este programa?



Este programa es un Sistema de Facturación llamado "AS" versión 2.6, desarrollado en Python con una interfaz gráfica basada en Tkinter y CustomTkinter. Está diseñado para gestionar operaciones comerciales, incluyendo:

- Gestión de inventario: Productos con códigos, precios, stock, proveedores y categorías.
- Clientes: Registro de datos personales, créditos y balances.
- Facturación: Creación de facturas con cálculo de IVA (configurable al 16%), tasa de cambio USD/BS, y manejo de pagos y créditos.
- Reportes: Gráficos y análisis de ventas usando Matplotlib.
- Usuarios: Roles de administrador y empleado con autenticación segura (contraseñas hasheadas).
- Persistencia: Datos almacenados en una base de datos SQLite (billing.db) y un archivo JSON (billing_data.json) para configuraciones adicionales.

Incluye funcionalidades como punto de venta, inventario, clientes, facturas y reportes en pestañas. Soporta exportación de datos, manejo de señales para guardado seguro y una UI moderna con iconos y logos. Dependencias principales: CustomTkinter, Matplotlib y Pillow.

1. Identificar: entradas, procesos y salidas

Desglosando el proyecto

1

Entradas

Usuario (interfaz gráfica):

Formularios para ingresar productos, clientes, facturas, pagos y usuarios.

Selección de productos y clientes en el punto de venta.

Campos para tasas de cambio, IVA, cantidades, precios, etc.

Botones para crear, editar, eliminar y buscar registros.

Login de usuario (nombre de usuario y contraseña).

Archivos:

Lectura de imágenes (logo).

Lectura y escritura de datos en archivos JSON

(billing_data.json).

Base de datos:

Lectura y escritura en la base de datos SQLite (productos, clientes, usuarios, facturas, pagos, documentos).



2

Procesos

Gestión de inventario: Alta, baja, modificación y consulta de productos.

Gestión de clientes: Registro, edición y consulta de clientes.

Facturación: Creación de facturas, cálculo de totales, IVA, manejo de crédito y pagos.

Autenticación: Validación de usuarios y roles (admin/empleador).

Reportes: Generación de reportes y estadísticas con gráficos.

Persistencia: Guardado y carga de datos en la base de datos y archivos.

Interfaz gráfica: Actualización dinámica de vistas, pestañas y widgets.

Manejo de errores: Validaciones, mensajes de error y manejo seguro de widgets.

1. Identificar: entradas, procesos y salidas

Desglosando el proyecto

3

Entradas

alidas

Interfaz gráfica:

Visualización de productos, clientes, facturas y reportes en pestañas/tablas.

Mensajes de confirmación, error y advertencia.

Gráficos y estadísticas de ventas.

Archivos:

Guardado de datos en archivos JSON y base de datos SQLite.

Base de datos:

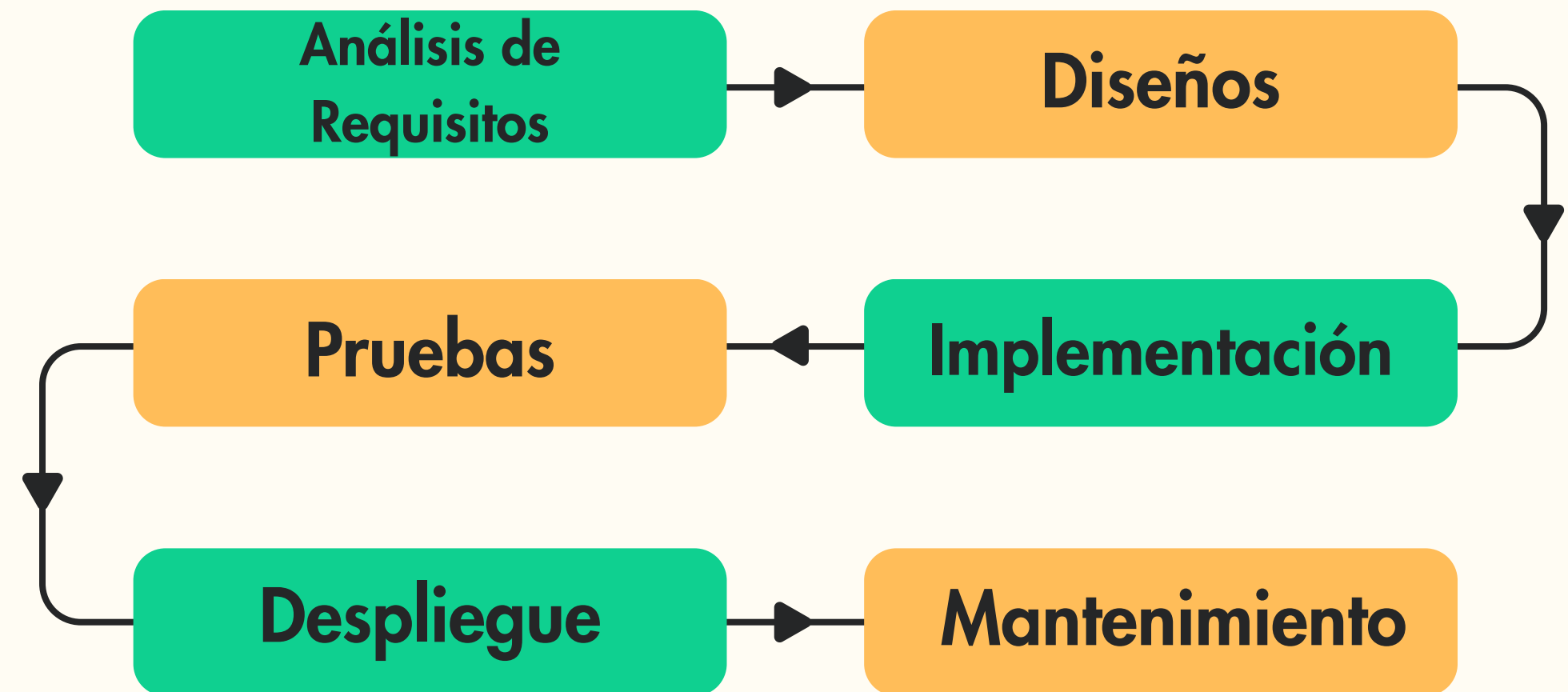
Actualización de registros de productos, clientes, facturas, pagos y usuarios.

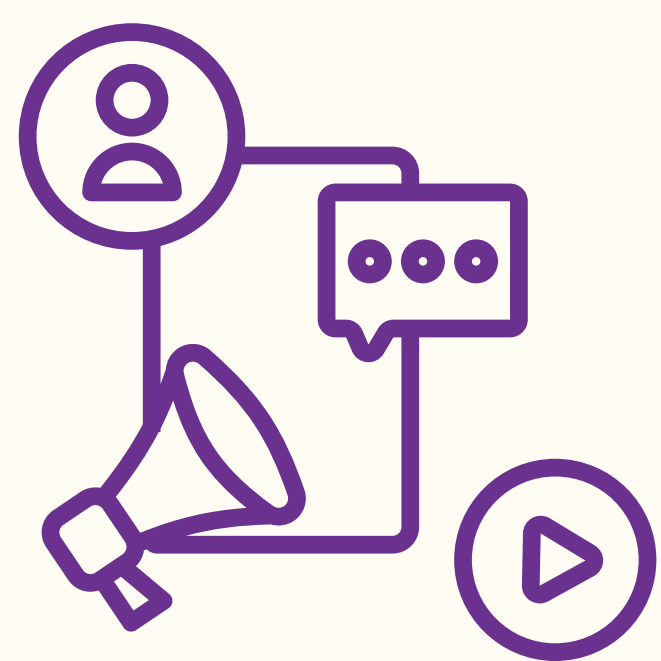
Reportes:

Visualización y posiblemente exportación de reportes y estadísticas..



2. Ciclo de vida del programa





3. TECNOLOGÍAS Y HERRAMIENTAS DE DESARROLLO EMPLEADAS EN EL PROGRAMA:

EL SISTEMA DE FACTURACIÓN AS V2.6 UTILIZA LAS SIGUIENTES TECNOLOGÍAS Y HERRAMIENTAS:

Lenguaje de Programación

Python: Lenguaje principal para toda la lógica, interfaz y procesamiento.

Librerías y Frameworks

Tkinter: Para la interfaz gráfica básica de usuario (widgets nativos).

CustomTkinter: Extensión moderna de Tkinter para una UI más atractiva y personalizable (temas, colores).

Pillow (PIL): Para cargar y manipular imágenes (e.g., logo de la aplicación).

Matplotlib: Para generar gráficos y reportes visuales (e.g., estadísticas de ventas).

sqlite3: Módulo estándar de Python para interactuar con la base de datos SQLite.

hashlib: Para encriptar contraseñas de usuarios (SHA-256).

datetime, json, os, signal: Módulos estándar para manejo de fechas, archivos JSON, sistema operativo y señales de cierre.

Base de Datos:

SQLite: Base de datos relacional embebida para almacenar productos, clientes, facturas, usuarios y pagos. No requiere servidor externo.

Herramientas de Desarrollo

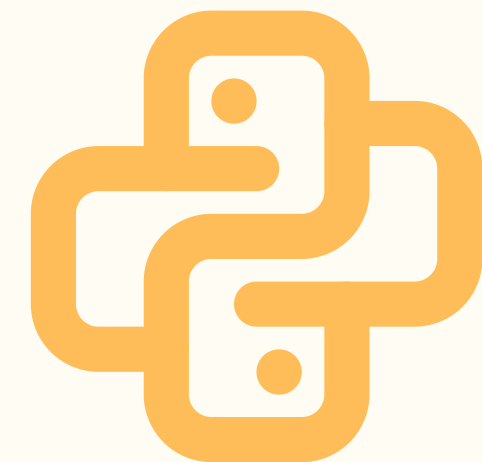
pip: Gestor de paquetes para instalar dependencias desde requirements.txt.

Entorno Virtual (.venv): Para aislar dependencias del proyecto (activado con PowerShell en Windows).

IDE/Editor: Compatible con VS Code (basado en el contexto del workspace), pero no especificado en el código.

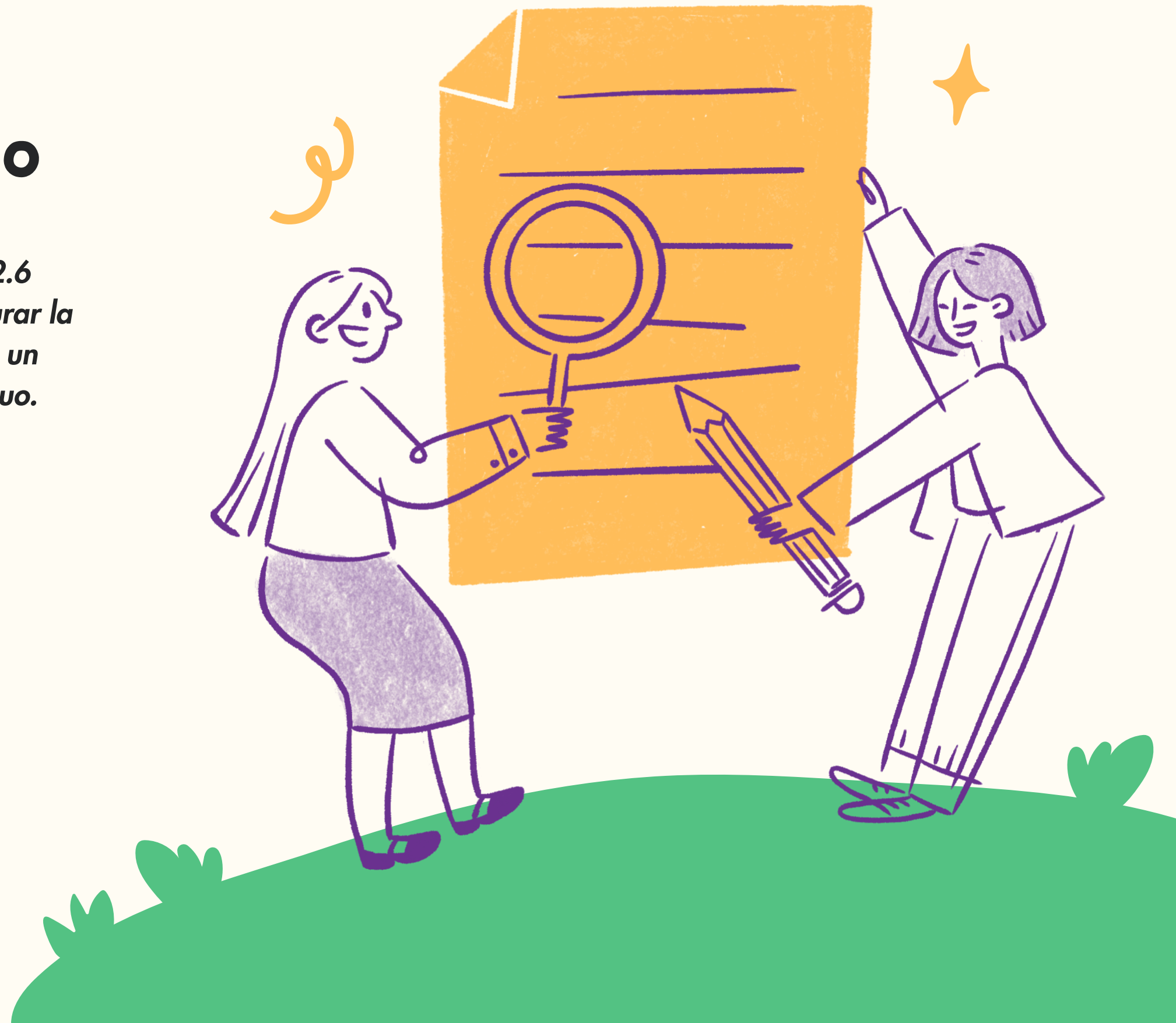
Pruebas: Scripts simples en Python (e.g., test_create_user.py) para validar funcionalidades básicas.

Otros



4. Plan de mantenimiento

Plan de Mantenimiento para el Sistema de Facturación AS v2.6
Este plan describe las actividades de mantenimiento para asegurar la estabilidad, seguridad y evolución del programa. Se basa en un enfoque proactivo, con revisiones periódicas y soporte continuo.

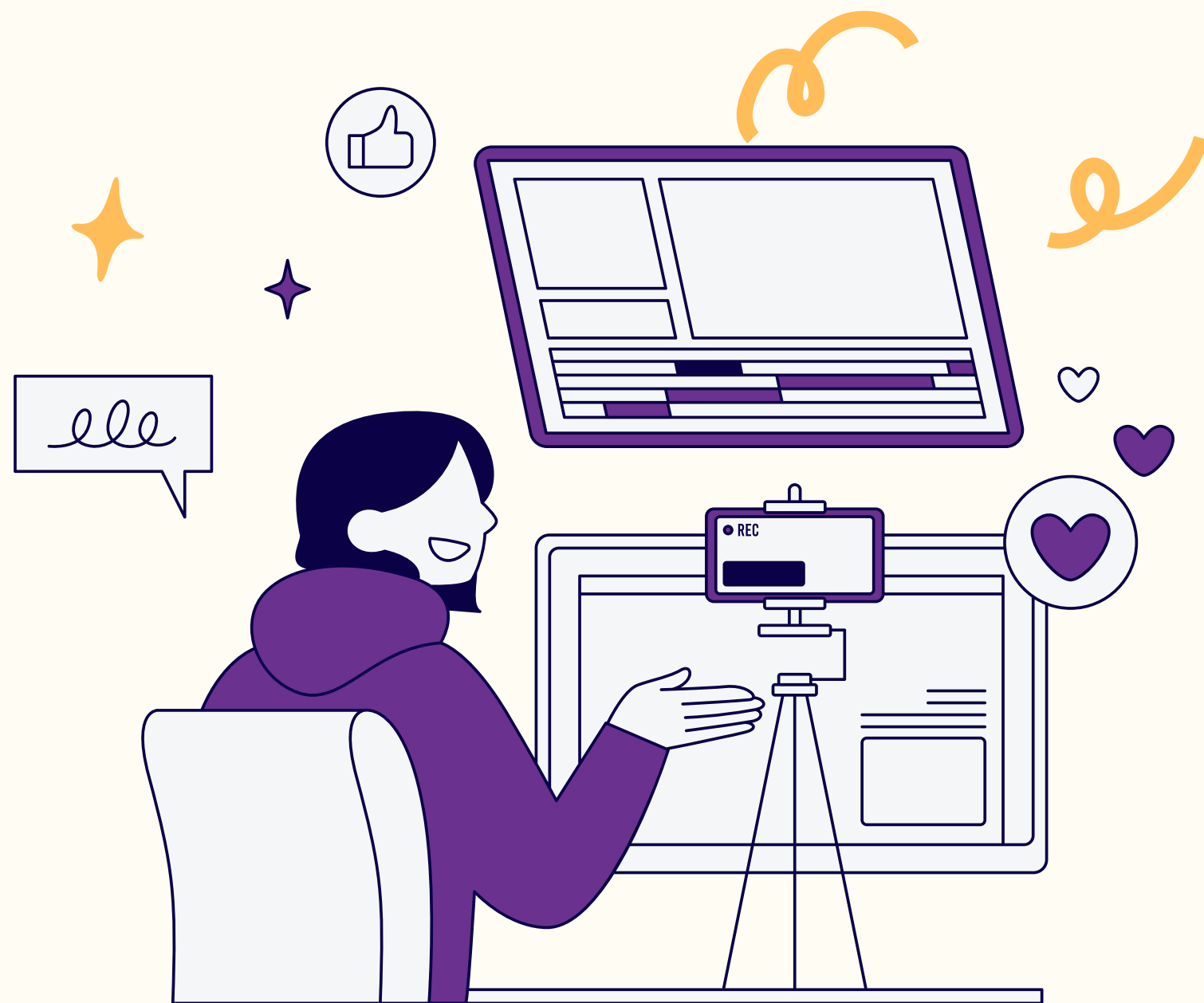


Objetivos del Plan



- Corregir errores y vulnerabilidades.
- Adaptar el software a cambios en el entorno (e.g., actualizaciones de Python o librerías).
- Mejorar funcionalidades basadas en feedback de usuarios.
- Prevenir fallos mediante backups y monitoreo.
- Extender la vida útil del software a través de versiones incrementales.





Tipos de Mantenimiento

Correctivo: Corrección de bugs reportados (e.g., errores en cálculos de IVA, fallos en UI con CustomTkinter).

Adaptativo: Actualizaciones para compatibilidad (e.g., nuevas versiones de Python, SQLite o librerías como Matplotlib).

Perfectivo: Mejoras y nuevas features (e.g., agregar exportación de reportes a PDF, optimizar permisos de empleados).

Preventivo: Reforzamiento de seguridad (e.g., mejoras en hashing de contraseñas, validaciones de entrada).

3. Frecuencia y Procedimientos

Diario/Semanal:

Monitoreo de logs y errores en ejecución.

Backups automáticos de la base de datos SQLite (billing.db) y archivos JSON (billing_data.json) al cerrar la aplicación (usando signal handlers existentes).

Mensual:

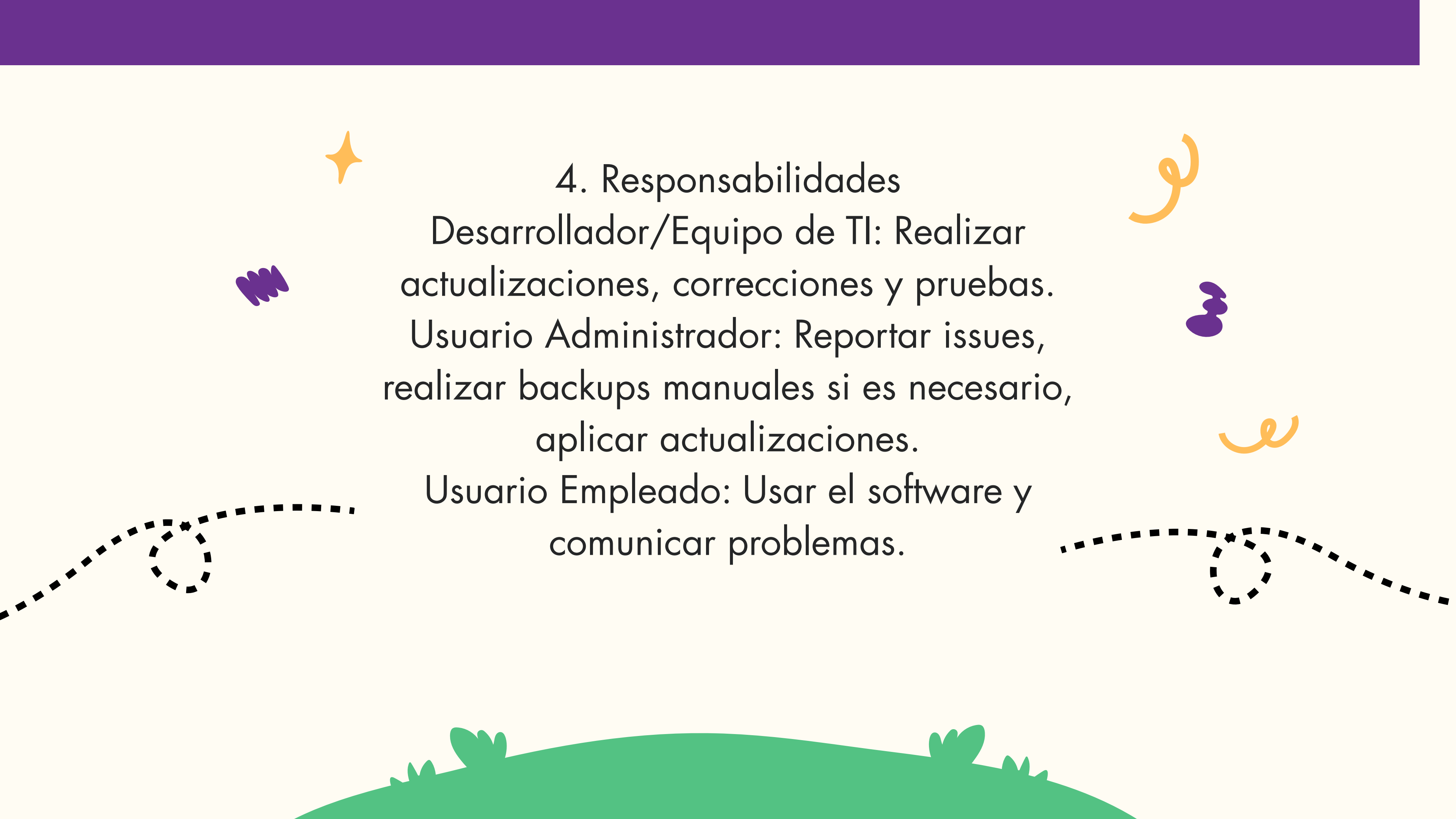
- - Revisión de dependencias: Ejecutar `pip list --outdated` y actualizar `requirements.txt` si es necesario.
 - Pruebas de funcionalidades críticas (e.g., facturación, login) usando scripts de test existentes.
 - Verificación de permisos y roles de usuario.

Trimestral:

- - Auditoría de seguridad: Revisar código para vulnerabilidades (e.g., inyección SQL en consultas SQLite).
 - Actualización de versiones: Liberar parches menores (e.g., v2.6.1) para correcciones.

Anual:

- - Mejoras mayores: Nueva versión (e.g., v2.7) con features solicitadas.
 - Migración de datos si cambia el esquema de BD.
 - Capacitación a usuarios sobre nuevas funcionalidades.

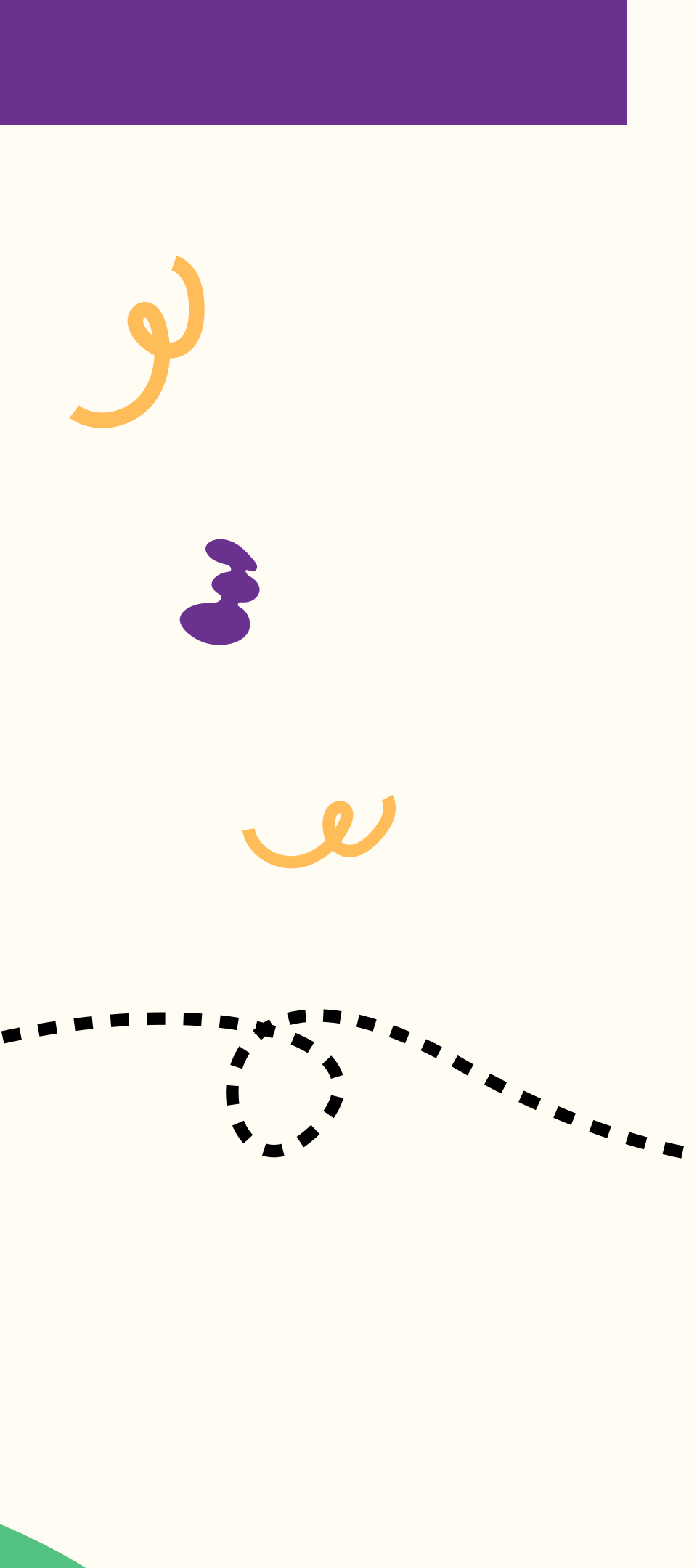


4. Responsabilidades

Desarrollador/Equipo de TI: Realizar actualizaciones, correcciones y pruebas.

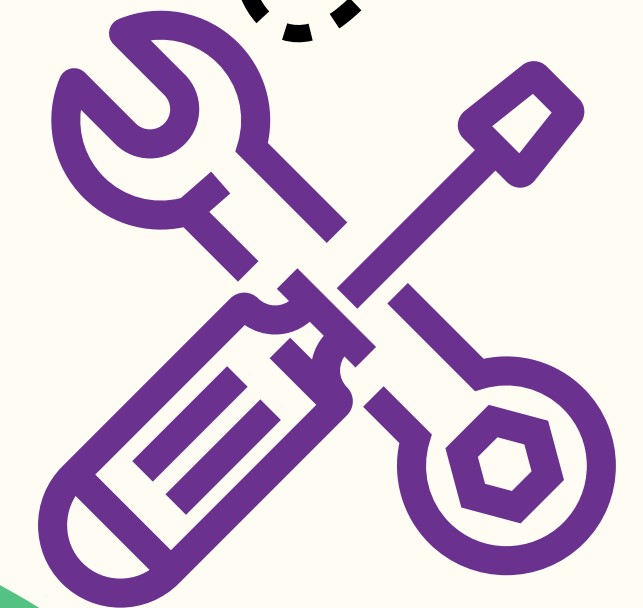
Usuario Administrador: Reportar issues, realizar backups manuales si es necesario, aplicar actualizaciones.

Usuario Empleado: Usar el software y comunicar problemas.



5. Herramientas y Recursos

- Herramientas: pip para dependencias, Git para control de versiones (si se implementa), SQLite Browser para inspeccionar BD.
- Recursos: Acceso a repositorio de código, entorno de pruebas (copia del .venv), documentación del código.
- Presupuesto: Estimado en 10-20% del costo inicial por año (para soporte y actualizaciones).



6. Riesgos y Mitigación

Riesgo: Pérdida de datos por fallos en BD.

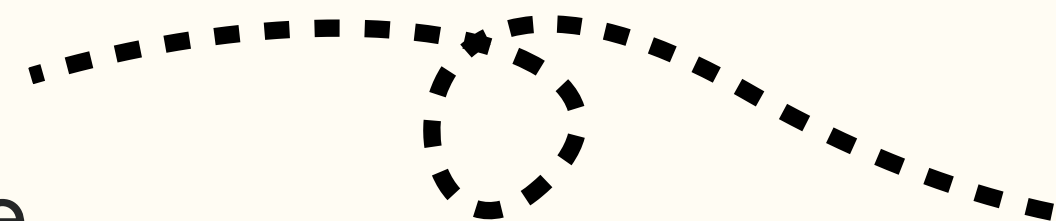
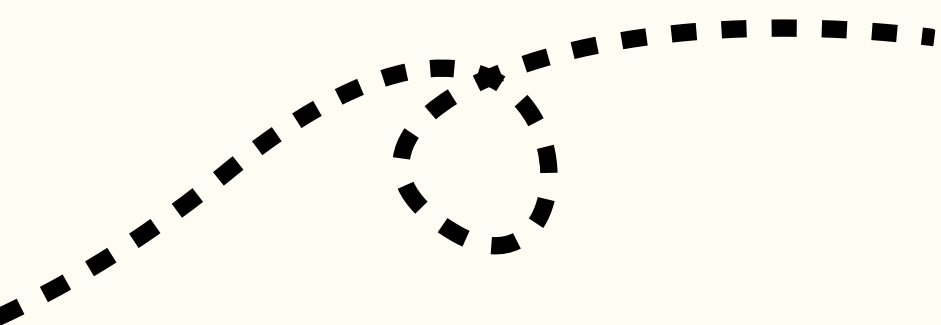
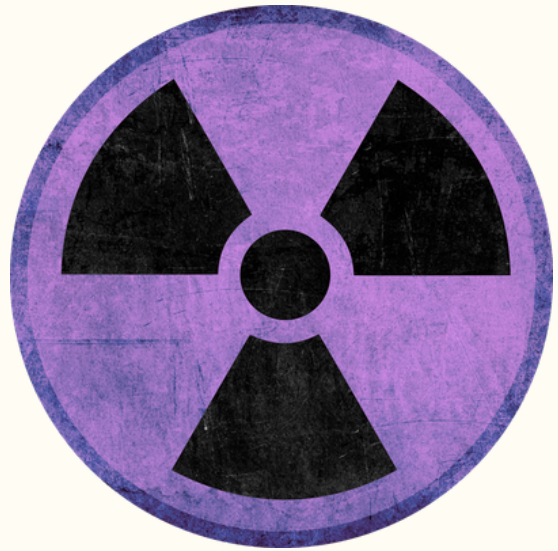
Mitigación: Backups automáticos y manuales; validaciones en código.

Riesgo: Incompatibilidades con actualizaciones de librerías.

Mitigación: Pruebas en entorno staging antes de desplegar.

Riesgo: Bugs no detectados.

Mitigación: Recopilar feedback y logs de usuarios.



7. Monitoreo y Evaluación

- Registrar todas las actividades en un log de mantenimiento.
- Evaluar efectividad anualmente:
Métricas como tiempo de resolución de issues y satisfacción de usuarios.
- Ajustar el plan basado en cambios en el negocio o tecnología.



ADIÓS

Por Andrés socorro

